

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI TP.HCM
VIỆN CÔNG NGHỆ THÔNG TIN VÀ ĐIỆN, ĐIỆN TỬ

DATA MINING

CHƯƠNG II: SELECTION

TS. Trần Thế Vinh



Phân loại dữ liệu

Theo cấu trúc:

1. **Dữ liệu có cấu trúc:** Dữ liệu có cấu trúc rõ ràng, biểu diễn dưới dạng bảng với các hàng – cột cố định. Ví dụ: SQL, file CSV, bảng excel.
2. **Dữ liệu bán cấu trúc:** Dữ liệu có cấu trúc linh hoạt, thường dưới dạng XML, JSON hay YAML, có cặp key – value nhưng không tuân thủ lược đồ cố định.
3. **Dữ liệu phi cấu trúc:** Dữ liệu không có cấu trúc xác định, bao gồm văn bản tự do, hình ảnh, âm thanh, video. Để xử lý, thường cần kỹ thuật NLP, Computer Vision hoặc xử lý tín hiệu.

Theo loại giá trị (Data Type)

1. **Numerical:** Giá trị số, chia thành rời rạc (số đếm) và liên tục (thang đo đo lường), ví dụ: số lượng bán, giá trị doanh thu.
2. **Categorical:** Dữ liệu phân loại, bao gồm *nominal* (không có thứ tự, ví dụ: giới tính, màu sắc) và *ordinal* (có thứ tự, ví dụ: xếp hạng thang điểm ‘thấp, trung bình, cao’).
3. **Textual:** Chuỗi ký tự, ví dụ: mô tả sản phẩm, bình luận khách hàng.
4. **Multimedia:** Hình ảnh, âm thanh, video, thường đòi hỏi xử lý đặc thù như trích xuất đặc trưng ảnh, tín hiệu âm thanh.
5. **Time Series:** Dữ liệu theo thời gian, ví dụ: giá cổ phiếu, nhiệt độ, sensor readings.

```
// example of data.json file
[
  {
    "id": 1,
    "firstName": "Thomas",
    "lastName": "Bækker",
    "address": {
      "streetName": "Tornelisevej",
      "streetNumber": "10",
      "city": "Solrød Strand"
    },
    "emergencyPhoneNumbers": ["+4530303030", "+4510203040"]
  },
  {
    "id": 2,
    "firstName": "Niels",
    "lastName": "Bohr",
    "address": {
      "streetName": "Flemsevej",
      "streetNumber": "31A",
      "city": "København"
    },
    "emergencyPhoneNumbers": []
  },
  ...
]
```

Chất lượng dữ liệu

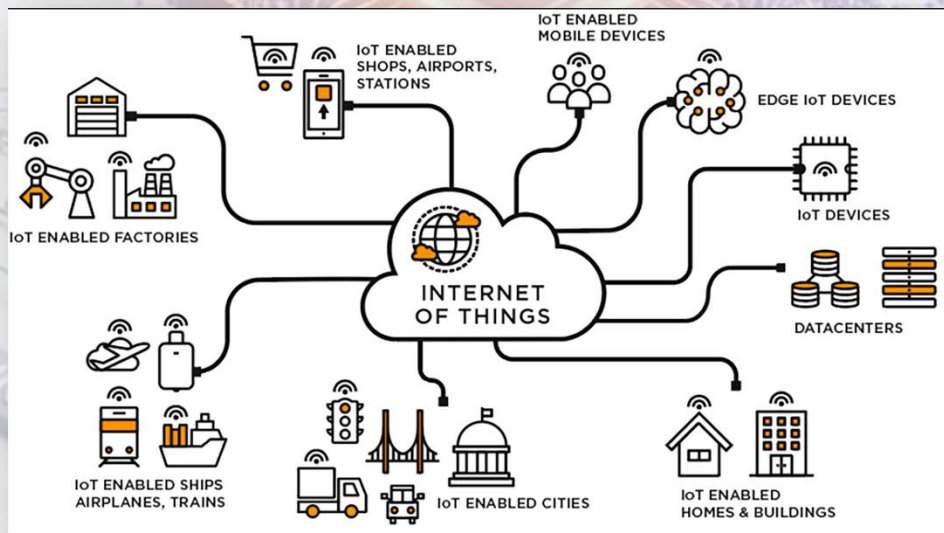
Đánh giá chất lượng dữ liệu là bước đầu tiên trong tiền xử lý. Một số độ đo chính:

1. **Completeness (Đầy đủ):** Tỷ lệ giá trị có mặt trên tổng số giá trị cần thu thập.
2. **Accuracy (Chính xác):** Mức độ phản ánh đúng thực tế của dữ liệu.
3. **Consistency (Nhất quán):** Dữ liệu không mâu thuẫn với nhau trong các nguồn hoặc bộ dữ liệu.
4. **Timeliness (Kịp thời):** Dữ liệu có được cập nhật đúng thời gian cần thiết không.
5. **Uniqueness (Không trùng lặp):** Khả năng duy nhất của mỗi bản ghi (không có bản sao).

Thu thập dữ liệu

Quá trình thu thập dữ liệu bao gồm:

1. **Xác định nguồn dữ liệu:** API, database, file lưu trữ, log hệ thống, thiết bị IoT.
2. **Thiết lập kết nối:** Sử dụng các công cụ như Python requests, thư viện kết nối CSDL (psycopg2, SQLAlchemy), hoặc Kafka Producer.
3. **Lập lịch thu thập:** Batch ingestion (theo lịch) hoặc streaming ingestion (realtime). Ví dụ: lên lịch cron job, hoặc sử dụng Apache Airflow.
4. **Ghi log và giám sát:** Theo dõi tỷ lệ lỗi, tốc độ ingest, độ trễ.



Trực quan hoá dữ liệu

Trực quan hóa là công cụ hỗ trợ **Khám phá dữ liệu (Exploratory Data Analysis)**. Giúp biến đổi dữ liệu → Hình ảnh → Con người hiểu dễ hơn.

1. Biểu đồ phân phối:

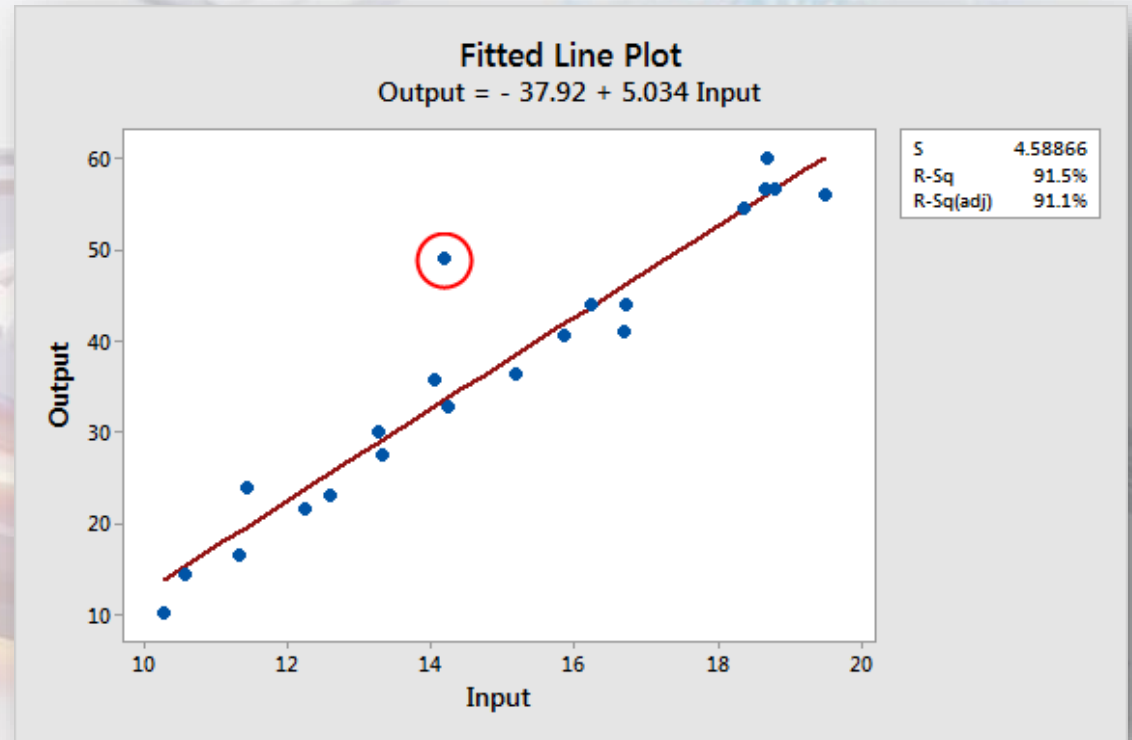
- Histogram: Phân bố giá trị số.
- Bar chart: Tần suất danh mục.

2. Biểu đồ quan hệ:

- Scatter plot: Tương quan giữa hai biến số.
- Line chart: Xu hướng theo thời gian.

3. Biểu đồ đặc biệt:

- Boxplot: Phát hiện ngoại lệ.
- Heatmap: Trực quan ma trận tương quan.



Việc trực quan hóa ban đầu giúp nhận diện **outliers**, **missing patterns**, và gợi ý **feature engineering**.

Trực quan hoá dữ liệu

Mục tiêu: Biến dữ liệu thành hình ảnh để phát hiện outliers, missing, phân bố, tương quan,....

Các bước cụ thể:

1. Kiểm tra phân phối giá trị số. (Sử dụng Histogram, Boxplot)
2. Phân tích tần suất giá trị phân loại (Bar chart, Countplot)
3. Phát hiện mối quan hệ giữa biến số (Scatter plot, Pairplot)
4. Phân tích theo thời gian (Line chart)
5. Phân tích tương quan biến số



Component Data Collection & Visualization trong Pipeline

Mục tiêu: Xây dựng module đầu tiên để **thu thập và khám phá dữ liệu**.

Work flow:

Nguồn dữ liệu → Collect → Clean sơ bộ → Load vào DataFrame → Visualization → Báo cáo khám phá

Input: Raw Data từ nhiều nguồn khác nhau (web, api, storage,..)

Process:

- Đọc dữ liệu bằng pandas
- Hiển thị thông tin bằng *df.info()*, *df.describe()*
- Trực quan bằng *matplotlib.pyplot* / *seaborn*

Output:

- Bảng mô tả dữ liệu
- Biểu đồ histogram, bar chart, scatter plot,..

Kết quả:

- Báo cáo thống kê tổng quan (mean, median, std)
- Biểu đồ phân phối gợi ý phân bố không chuẩn hoặc tồn tại *outliers*

Component Data Collection & Visualization trong Pipeline

Phân tích từng loại dữ liệu:

1. CSV / Structured Tabular Data (Dữ liệu có cấu trúc bảng)

- Nguồn: File, Database, API, Synthetic generator.
- Đọc: `pandas.read_csv`, `sqlalchemy`,..
- Thông tin khám phá:
 - `df.info()`, `df.describe()`
 - Histogram, Boxplot, Correlation Heatmap

2. Text / Log / Document Data

- Nguồn: Log hệ thống, file, .txt, .json, HTML, Synthetic generator
- Xử lý: Tokenize, remove stopwords, TF-IDF / BERT
- Trực quan hoá:
 - Bar chart từ word frequency
 - WordCloud
 - Sentence length histogram

3. Video Data

- **Nguồn:** Camera, Stream, File
- **Xử lý:**
 - Trích frame, Optical Flow
 - Trích âm thanh, metadata
- **Trực quan hóa:**
 - Frame grid
 - Timeline event
 - Histogram độ dài, FPS

4. Image Data

- **Nguồn:** File hệ thống, Web, synthetic images.
- **Xử lý:**
 - Resize, normalize
 - Trích đặc trưng: color histogram, edge density
- **Trực quan hóa:**
 - Image grid
 - Histogram cường độ sáng
 - t-SNE / PCA ảnh giảm chiều

5. Audio / Voice Data

- **Nguồn:** Mic, File .wav, .mp3, synthetic sine waves
- **Xử lý:**
 - Extract MFCC, spectral contrast
 - Chuyển waveform → feature vector
- **Trực quan hóa:**
 - Waveform plot
 - Spectrogram
 - Histogram độ dài, tần số

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI TP.HCM
VIỆN CÔNG NGHỆ THÔNG TIN VÀ ĐIỆN, ĐIỆN TỬ

DATA MINING

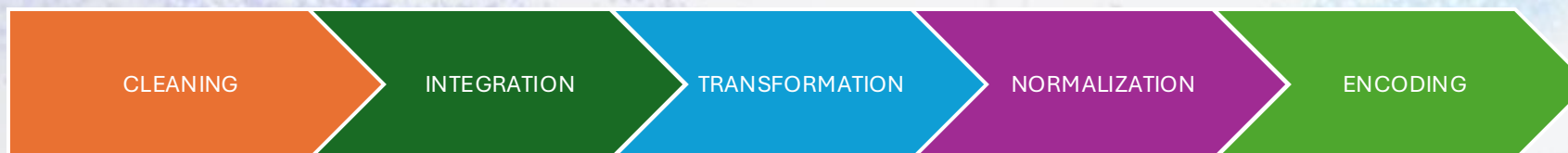
CHƯƠNG III: PREPROCESSING

TS. Trần Thế Vinh

Tiền xử lý dữ liệu

Tiền xử lý dữ liệu là giai đoạn trọng yếu trong chu trình khai phá dữ liệu. Chất lượng của mô hình phân tích và học máy phụ thuộc đáng kể vào cách dữ liệu được xử lý trước.

Giai đoạn này bao gồm các bước chính:



Về mặt bản chất, tiền xử lý dữ liệu là quá trình **biến đổi không gian dữ liệu đầu vào** để mô hình hóa hiệu quả hơn trong không gian đặc trưng.

Tại sao cần tiền xử lý?

- Dữ liệu thực tế thường không hoàn chỉnh, không đồng nhất, chứa nhiễu.
- Chất lượng dữ liệu ảnh hưởng **trực tiếp** đến kết quả mô hình.
- Đây là bước bắt buộc trong quy trình **CRISP-DM (giai đoạn Preparation)** và **KDD (Preprocessing & Transformation)**.

Xây dựng module “Preprocessing” trong Pipeline

Mục tiêu: Xây dựng module tiền xử lý dữ liệu ”preprocessing”.

Work flow:

Raw Data → Cleaning → Integration → Transformation → Encoding → Ready Data

Input: Raw Data.

Cleaning → Integration → Transformation → Encoding :

- Loại bỏ hoặc sửa chữa những điểm khuyết trong dữ liệu
- *Thống nhất dữ liệu từ nhiều nguồn thành một tập hợp nhất quán*
- *Đưa dữ liệu về dạng phù hợp cho phân tích hoặc huấn luyện.*
- *Chuyển dữ liệu phi số (text, image,..) thành số học được mô hình hiểu.*

Output:

- Một tập dữ liệu “sạch”, ”chuẩn” và ”tối ưu” (Ready Data).

Kết quả:

- Dữ liệu sau khi tiền xử lý sẽ đi qua pipeline:
 - Phân lớp (Classification)
 - Gom cụm (Clustering)
 - Khai thác luật kết hợp
 - Dự báo (Forecasting)..
PREPROCESSING

Làm sạch dữ liệu (Data Cleaning)

1. Structured Data:

- **Mục tiêu:**
 - Xử lý giá trị thiếu (missing values).
 - Xử lý dữ liệu không hợp lệ hoặc nhiễu (invalid/noisy data).
 - Phát hiện và loại bỏ điểm ngoại lệ (outlier)
- **Kỹ thuật xử lý phổ biến:**

Vấn đề	Phương pháp
Missing values	dropna(), fillna(method='mean'), nội suy
Ouliers	Z-score, IQR
Duplicates	drop_duplicates()
Inconsistent Types	astype() chuyển đổi kiểu dữ liệu
Domain Constrains	Áp điều kiện ràng buộc

- **Công cụ/Thư viện:**
 - Python/Pandas: dropna, fillna, describe, astype, duplicated
 - Scikit-learn: SimpleImputer, KNNImputer, RobustScaler.

Làm sạch dữ liệu (Data Cleaning)

Mô hình toán học:

a. Xử lý dữ liệu thiếu (Missing Data)

Giả sử một thuộc tính $x \in \mathbb{R} \cup \{NaN\}$

Phương pháp:

- Loại bỏ (deletion): Nếu thiếu toàn dòng $x_i = NaN \Rightarrow drop(x_i)$
- Thay thế:
 - Trung bình $x_i = \bar{x} = \frac{1}{n} \sum_{j=1}^n x_j$
 - Trung vị $x_i = median(x)$
 - Giá trị phổ biến nhất (mode) nếu dữ liệu rời rạc.
- Dự đoán bằng mô hình: sử dụng hồi quy hoặc cây quyết định để ước lượng giá trị tối thiểu

b. Xử lý dữ liệu nhiễu (Noise)

- Phát hiện bằng thống kê:
 - Boxplot: phát hiện outlier qua IQR (Interquartile Range) $= Q_3 - Q_1$
 $Outlier \text{ if } x > Q_3 + 1.5 \times IQR \text{ or } x < Q_1 - 1.5 \times IQR$
 - Z-score:
 - Đo độ lệch chuẩn so với trung bình:
$$z = \frac{x_i - \mu}{\sigma}$$
Nếu $|z| > 3$ thì được coi là outlier (theo phân phối chuẩn)
- Làm mượt (smoothing):
 - Smoothing by binning
 - Smoothing by regression
 - Moving average

Làm sạch dữ liệu (Data Cleaning)

2. Dữ liệu văn bản (Semi-Structured & Unstructured Data)

- **Mục tiêu:**
 - Chuẩn hoá và giảm nhiễu ngôn ngữ
 - Tăng khả năng học mô hình bằng cách chuẩn hoá cú pháp, từ vựng
- **Kỹ thuật xử lý phổ biến:**

Kỹ thuật	Mô tả
Lowercasing	"Khoa Học Dữ Liệu" → "khoa học dữ liệu"
Sửa lỗi Unicode	Sửa các lỗi mã hóa tiếng Việt (dấu sai)
Chuẩn hóa khoảng trắng	Xoá khoảng trắng thừa, tab, xuống dòng
Ký tự đặc biệt	!~#\$%^&*() → dùng regex: <code>re.sub(r'^\w\s', "", text)</code>
Stopword tiếng Việt	"và", "là", "một", "những", v.v.

- **Công cụ/Thư viện:**
 - **Python:** `re`, `unicodedata`, `html.parser`
 - **NLTK, spaCy:** `stopwords`, `lemmatizer`
 - **BeautifulSoup:** `HTML cleaner`

Làm sạch dữ liệu (Data Cleaning)

3. Dữ liệu hình ảnh (Image Data)

- **Mục tiêu:**
 - Xử lý lỗi ảnh, ảnh nhiễu, không đồng nhất kích thước
 - Tăng chất lượng dữ liệu đầu vào cho mô hình thị giác máy tính.
- **Kỹ thuật xử lý phổ biến:**

Kỹ thuật	Mô tả
Kiểm tra kích thước	Lọc ảnh quá nhỏ hoặc hỏng
Resize	Chuẩn hóa về kích thước cố định
Gaussian Blur	Khử nhiễu (noise reduction), làm mượt
Histogram Equalization	Cân bằng ánh sáng ảnh

- **Công cụ/Thư viện:**
 - **OpenCV:** cv2.imread, cv2.resize, cv2.GaussianBlur
 - **Pillow (PIL):** Image.open(), resize()
 - **Scikit-Image**

Làm sạch dữ liệu (Data Cleaning)

4. Dữ liệu âm thanh (Audio Data)

- **Mục tiêu:**
 - Xử lý tiếng ồn nền và đoạn không tiếng.
 - Chuẩn bị dữ liệu âm thanh cho trích xuất đặc trưng hoặc nhận dạng giọng nói.
- **Kỹ thuật xử lý phổ biến:**

Kỹ thuật	Mô tả
Silence Removal	Phát hiện đoạn yên lặng bằng năng lượng tín hiệu
Noise Reduction	Dùng Spectral Gating hoặc Wiener Filter
Normalization	Cân bằng biên độ âm thanh

- **Công cụ/Thư viện:**
 - Librosa, Pydub, Soundfile
 - Noisereduce, scipy.signal

Làm sạch dữ liệu (Data Cleaning)

5. Dữ liệu video (Video Data)

- **Mục tiêu:**
 - Phát hiện và loại bỏ các frame lỗi.
 - Chuẩn hoá định dạng, tốc độ khung hình (FPS), kích thước khung.
- **Kỹ thuật xử lý phổ biến:**

Kỹ thuật	Mô tả
Frame Extraction	Tách từng frame ra để xử lý
Frame Interpolation	Chuẩn hóa FPS
Frame Quality Check	Tính độ sắc nét (ví dụ sử dụng: Laplacian variance)
Denoising	Denoise toàn video hoặc từng frame

- **Công cụ/Thư viện:**
 - **OpenCV, ffmpeg, moviepy**
 - **Scikit-video, Deep Learning models for denoising**

Tích hợp dữ liệu (Data Integration)

Mục tiêu:

- Kết hợp dữ liệu từ nhiều nguồn khác nhau thành một tập dữ liệu thống nhất.
- Loại bỏ mâu thuẫn dữ liệu, trùng lặp, và xử lý sai lệch thời gian.
- Chuẩn hoá lại định dạng, schema và cấu trúc dữ liệu

1. Dữ liệu bảng(Structured Data)

Kỹ thuật chính:

- **Merge / Join** dựa trên **khóa chung** (ID người dùng, mã giao dịch, timestamp,...)
- Xử lý các loại join: inner, left, right, outer
- **Chuẩn hóa schema**: tên cột, kiểu dữ liệu, định dạng thời gian

Các bước xử lý:

1. **Đọc dữ liệu** từ nhiều nguồn (CSV, SQL, API,...)
2. **Chuẩn hóa cột**:
 - Đồng nhất kiểu ngày (datetime)
 - Cột định danh cùng kiểu (int, str)
3. **Thực hiện join**:
 - `df = pd.merge(df1, df2, on="user_id", how="inner")`
4. **Loại bỏ bản ghi trùng / thiếu** sau khi tích hợp
5. **Ghi log số lượng bản ghi thay đổi** để kiểm tra

Kết quả thu được một bảng duy nhất, đầy đủ thông tin, không conflict, đúng kiểu dữ liệu

Tích hợp dữ liệu (Data Integration)

Mục tiêu:

- Kết hợp dữ liệu từ nhiều nguồn khác nhau thành một tập dữ liệu thống nhất.
- Loại bỏ mâu thuẫn dữ liệu, trùng lặp, và xử lý sai lệch thời gian.
- Chuẩn hoá lại định dạng, schema và cấu trúc dữ liệu

2. Dữ liệu văn bản(Text, Logs)

Kỹ thuật chính:

- Gộp log từ nhiều hệ thống
- Gom nhóm theo **người dùng** hoặc **phiên làm việc (session)**
- **Sắp xếp thời gian**, loại bỏ bản ghi không hợp lệ

Các bước xử lý:

1. **Thu thập log từ các hệ thống** (web, mobile, API logs)
2. **Gộp log** từ nhiều file/dịch vụ vào một DataFrame hoặc MongoDB collection
3. **Phân nhóm:**
 - Theo *session_id*, *user_id*
4. **Sắp xếp theo timestamp:**
 - `df.sort_values(by=["session_id", "timestamp"], inplace=True)`
5. **Gắn nhãn (labeling)** nếu phục vụ học máy

Kết quả thu được tài liệu log dạng chuẩn, sắp xếp đúng thời gian, dễ phân tích và trích xuất session

Tích hợp dữ liệu (Data Integration)

Mục tiêu:

- Kết hợp dữ liệu từ nhiều nguồn khác nhau thành một tập dữ liệu thống nhất.
- Loại bỏ mâu thuẫn dữ liệu, trùng lặp, và xử lý sai lệch thời gian.
- Chuẩn hoá lại định dạng, schema và cấu trúc dữ liệu

3. Dữ liệu ảnh và âm thanh(Image & Audio)

Kỹ thuật chính:

- **Tổ chức thư mục nhất quán** theo lớp (label)
- **Kết hợp metadata** từ CSV/JSON chứa nhãn và mô tả
- **Đồng bộ hóa label**, kiểm tra sự tồn tại file

Các bước xử lý:

1. **Quét thư mục ảnh/âm thanh:**
 - `from glob import glob image_paths = glob("dataset/images/**/*.*", recursive=True)`
2. **Đọc metadata** từ `labels.csv` hoặc `labels.json`
3. **Match file với label:**
 - Dựa theo `filename`, `id`, hoặc `hash`
 - Kiểm tra khớp `file` thực tế và `metadata`
4. **Tổ chức lại cấu trúc lưu trữ:**
 - Ví dụ: `dataset/images/{label}/{filename}`
 - Tạo symbolic link hoặc copy file

Kết quả thu được thư mục dữ liệu sạch, đồng bộ nhãn, có thể dùng cho huấn luyện mô hình

Tích hợp dữ liệu (Data Integration)

Mục tiêu:

- Kết hợp dữ liệu từ nhiều nguồn khác nhau thành một tập dữ liệu thống nhất.
- Loại bỏ mâu thuẫn dữ liệu, trùng lặp, và xử lý sai lệch thời gian.
- Chuẩn hoá lại định dạng, schema và cấu trúc dữ liệu

4. Dữ liệu videos(Video Data)

Kỹ thuật chính:

- **Tách audio/video stream** nếu cần
- Đồng bộ hóa timestamp giữa hình và tiếng
- Tích hợp thông tin nhãn và mô tả từ file ngoài

Các bước xử lý:

1. Tách stream nếu cần:

- `ffmpeg -i video.mp4 -vn audio.wav # Tách audio`
- `ffmpeg -i video.mp4 -an video_only.mp4 # Tách video không tiếng`

2. Lấy metadata từ video: độ dài, fps, timestamp

- `ffprobe -v quiet -print_format json -show_format -show_streams video.mp4`

3. Đồng bộ hóa label/metadata:

- Match theo `video_id`, `timestamp`

4. Kiểm tra lỗi mất tiếng/hình, đồng bộ lại nếu sai lệch

5. Ghi lại tập tin dạng chuẩn (MP4 + JSON metadata)

Kết quả thu được dữ liệu video không bị mất tiếng/hình, có metadata đồng bộ theo thời gian

Chuyển đổi dữ liệu (Transformation)

Mục tiêu:

- Chuyển dữ liệu về dạng phù hợp hơn cho mô hình học máy.
- Giảm nhiễu, mức độ chênh lệch giá trị, và tăng tính phân biệt của đặc trưng

1. Dữ liệu văn bản (Text Data)

Kỹ thuật chuyển đổi:

- **Chuẩn hoá thang đo:** Đưa tất cả giá trị về cùng một không gian tỷ lệ, tránh ảnh hưởng bởi thang đo khác nhau

1. Min – Max scaling:

- Biến đổi về khoảng $[0,1]$:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \in [0,1]$$

- Áp dụng khi dữ liệu không có phân phối chuẩn.

2. Standardization (Z-score Scaling):

- Chuẩn hoá về trung bình 0, phương sai 1:

$$x' = \frac{x - \mu}{\sigma}, \mu = \text{mean}, \sigma = \text{std} \Rightarrow$$

$$x' \sim N(0,1)$$

- Phù hợp khi mô hình học máy giả định phân phối chuẩn

3. Log-transform:

- Giảm skewness cho các biến có phân phối lệch (ví dụ, thu nhập, số lượng,..):

$$x' = \log(1 + x)$$

- **Rời rạc hoá (Discretization):** Biến đổi thuộc tính liên tục thành các nhóm rời rạc nhằm giảm nhiễu, dễ trực quan hoá, và tăng tính giải thích của mô hình.

1. Equal-width: Chia đều khoảng giá trị thành k đoạn bằng nhau ($[0-9]$, $[10-19]$, $[20-29]$,...)

2. Equa-frequency(quantile binning): Mỗi đoạn chứa số lượng mẫu gần bằng nhau (phần vị 25%, 50%, 75%)

3. K-means binning: Gom cụm 1D trên giá trị

4. Decision tree binning: Chia theo điểm cắt tối ưu dựa trên entropy/gini

5. Manual binning: Tự xác định theo domain knowledge (trẻ, trung niên, cao tuổi)

Chuyển đổi dữ liệu (Transformation)

Mục tiêu:

- Chuyển dữ liệu về dạng phù hợp hơn cho mô hình học máy.
- Giảm nhiễu, mức độ chênh lệch giá trị, và tăng tính phân biệt của đặc trưng

2. Dữ liệu văn bản (Text Data)

Các bước phổ biến:

1. Tokenization:

- Tách câu thành từ hoặc từ thành token:

2. Stemming:

- Loại bỏ các hậu tố, tiền tố đơn giản để đưa về gốc gần nhất

3. Lemmatization:

- Đưa từ về dạng chuẩn đúng ngữ pháp và từ vựng, có nghĩa theo từ điển.

4. Lowercasing:

- Đồng nhất chữ viết hoa và thường để tránh phân biệt vô nghĩa.

5. Stopword removal:

- Loại bỏ các từ không mang nhiều thông tin như: là, và, ờ, đang,...

6. POS tagging:

- Xác định vai trò của từ trong câu: danh từ, động từ, trạng từ, tính từ,...
- Giúp cho lemmatization chính xác hơn.

Chuyển đổi dữ liệu (Transformation)

Mục tiêu:

- Chuyển dữ liệu về dạng phù hợp hơn cho mô hình học máy.
- Giảm nhiễu, mức độ chênh lệch giá trị, và tăng tính phân biệt của đặc trưng

3. Dữ liệu hình ảnh (Image Data)

Các kỹ thuật chuyển đổi:

1. **Normalization – Chuẩn hoá pixel:** Đưa giá trị pixel về cùng thang đo để tránh mô hình bị ảnh hưởng bởi biến thiên độ sáng. Giúp tối ưu nhanh hơn, mô hình hội tụ tốt hơn.
 - Từ $[0,255] \rightarrow [0,1] : \frac{x}{255}$
 - Từ $[0,255] \rightarrow [-1,1] : \frac{x-127,5}{127,5}$
 - Z-score (mean/std): $\frac{x-\mu}{\sigma}$
2. **Augmentation – Tăng dữ liệu bằng biến đổi:** Tạo thêm dữ liệu mới từ dữ liệu gốc để tăng độ đa dạng hình ảnh, giảm overfitting, mô phỏng các tình huống thực tế (Flip, Rotate, Brightness, Crop, Noise, Resize, Blur,...)
 - Một bức ảnh duy nhất $\rightarrow$ Hàng chục biến thể ảnh khác nhau
 - Mô hình dễ overfit $\rightarrow$ Học được nhiều tính chất tổng quát hơn.

Chuyển đổi dữ liệu (Transformation)

Mục tiêu:

- Chuyển dữ liệu về dạng phù hợp hơn cho mô hình học máy.
- Giảm nhiễu, mức độ chênh lệch giá trị, và tăng tính phân biệt của đặc trưng

4. Dữ liệu âm thanh (Audio Data)

Các kỹ thuật chuyển đổi:

1. Chuẩn hoá định dạng (Format Standardization).

- Đảm bảo tất cả các file âm thanh có cùng định dạng để dễ xử lý.
 - Chuyển sang định dạng chuẩn như .wav, .pcm, .flac (tốt cho lưu trữ không mất dữ liệu)
 - Loại bỏ metadata không cần thiết (ID3 tags,..)

2. Sampling Rate và Channels:

- Sampling Rate Normalization: Thường chuẩn hoá về 16 kHz hoặc 16.0000 samples/s.
- Channel: Chuyển toàn bộ âm thanh sang mono để tránh xử lý kênh stereo gây nhiễu.

3. Voice Activity Detection (VAD):

- Xác định và giữ lại đoạn có giọng nói (speech) → loại bỏ đoạn không có tiếng (khoảng lặng).

4. Noise Reduction (giảm nhiễu)

- Loại bỏ nhiễu môi trường, cải thiện chất lượng âm thanh trước khi trích xuất đặc trưng.

Chuyển đổi dữ liệu (Transformation)

Mục tiêu:

- Chuyển dữ liệu về dạng phù hợp hơn cho mô hình học máy.
- Giảm nhiễu, mức độ chênh lệch giá trị, và tăng tính phân biệt của đặc trưng

5. Dữ liệu Video (Video Data)

Các kỹ thuật chuyển đổi:

1. Trích Frame:

- Tách video thành ảnh tĩnh theo FPS để xử lý ảnh thông thường hoặc thời gian cố định giúp mô hình nhìn thấy diễn biến theo thời gian.

2. Trích Optical Flow:

- Mô tả chuyển động giữa các frame kế tiếp.
- Cung cấp đặc trưng về vận tốc, hướng di chuyển, rất quan trọng trong hành vi, hành động

3. Chuyển video thành tensor 3D hoặc 4D:

- Đưa video vào mô hình học sâu (CNN 3D, RNN, Transformer).
- Biểu diễn dữ liệu video như 1 khối đa chiều

Mã hoá dữ liệu (Encoding)

Mục tiêu:

- Trong giai đoạn tiền xử lý, encoding là quá trình chuyển đổi các dữ liệu đầu vào (thường có dạng rời rạc, phi cấu trúc) thành dạng vector số học nhằm mục đích:
 - Làm dữ liệu trở nên hiểu được đối với các thuật toán máy học hoặc khai phá dữ liệu.
 - Bảo toàn hoặc rút trích được những ngữ nghĩa và cấu trúc nội tại của dữ liệu
 - Chuẩn hoá định dạng biểu diễn giữa các loại dữ liệu khác nhau (cấu trúc, bán cấu trúc, phi cấu trúc)

1. Encoding dữ liệu bảng(Structured Tabular Data)

Biến đổi các trường dạng phân loại (categorical) hoặc văn bản rời rạc thành các giá trị số để đưa vào mô hình

Các kỹ thuật chuyển đổi:

1. Label Encoding:

- Gán số nguyên cho mỗi nhãn phân loại.
- Ví dụ: [active, broken, repair, sold] $\rightarrow$ [0, 1, 2, 3]

2. One-hot Encoding:

- Tạo vector binary cho mỗi giá trị của biến.
- Ví dụ: "active" $\rightarrow$ [1, 0, 0, 0], "broken" $\rightarrow$ [0, 1, 0, 0]

3. Ordinal Encoding:

- Gán số theo thứ tự có ý nghĩa.
- Ví dụ: ["thấp", "trung bình", "cao"] $\rightarrow$ [1, 2, 3]

4. Binary Encoding/ Hash Encoding:

- Phù hợp khi số lượng nhãn rất lớn.
- Dùng trong bài toán xử lý metadata quy mô lớn (ID, zip code,...)

Mã hoá dữ liệu (Encoding)

Mục tiêu:

- Trong giai đoạn tiền xử lý, encoding là quá trình chuyển đổi các dữ liệu đầu vào (thường có dạng rời rạc, phi cấu trúc) thành dạng vector số học nhằm mục đích:
 - Làm dữ liệu trở nên hiểu được đối với các thuật toán máy học hoặc khai phá dữ liệu.
 - Bảo toàn hoặc rút trích được những ngữ nghĩa và cấu trúc nội tại của dữ liệu
 - Chuẩn hoá định dạng biểu diễn giữa các loại dữ liệu khác nhau (cấu trúc, bán cấu trúc, phi cấu trúc)

2. Encoding Text (Text Data)

Biến đổi chuỗi ký tự (từ, câu, đoạn) thành vector số phản ánh ngữ nghĩa.

Các kỹ thuật chuyển đổi:

1. Bag of Words(BoW):

- Vector đếm tần suất từ trong câu.
- Không xét đến ngữ cảnh

2. TF-IDF (Term Frequency – Inverse Document Frequency):

- Cân bằng giữa tần suất từ trong văn bản và độ hiếm trong toàn tập.

3. Word Embedding:

- Word2Vec:
 - Học vector từ trong ngữ cảnh (Skip-gram, CBOW)
 - Vector cố định cho mỗi từ
- FastText:
 - Xét n-gram ký tự → hỗ trợ từ chưa gặp (OOV).

4. Contextual Embedding:

- BERT (Bidirectional Encoder Representations from Transformers):
 - Tạo vector biểu diễn từng từ phụ thuộc vào ngữ cảnh đầy đủ
 - Vector có thể đại diện cho câu hoặc đoạn.

Mã hoá dữ liệu (Encoding)

Mục tiêu:

- Trong giai đoạn tiền xử lý, encoding là quá trình chuyển đổi các dữ liệu đầu vào (thường có dạng rời rạc, phi cấu trúc) thành dạng vector số học nhằm mục đích:
 - Làm dữ liệu trở nên hiểu được đối với các thuật toán máy học hoặc khai phá dữ liệu.
 - Bảo toàn hoặc rút trích được những ngữ nghĩa và cấu trúc nội tại của dữ liệu
 - Chuẩn hoá định dạng biểu diễn giữa các loại dữ liệu khác nhau (cấu trúc, bán cấu trúc, phi cấu trúc)

3. Encoding hình ảnh (Image Data)

Trích xuất đặc trưng không gian từ ảnh, chuyển đổi thành vector cố định.

Các kỹ thuật chuyển đổi:

1. Trích đặc trưng bằng CNN:

- Dùng mô hình như ResNet, EfficientNet, MobileNet,...
- Đầu ra là tensor đặc trưng không gian (C, H, W)

2. Flatten hoặc Global Pooling:

- Chuyển đổi tensor thành vector cố định
- Flatten: trải toàn bộ tensor thành một chiều
- Global Average Pooling: lấy trung bình trên không gian.

3. Pretrained Embedding:

- Sử dụng mô hình đã huấn luyện (ImageNet) để lấy đặc trưng ảnh.
- Ví dụ: ResNet50 → trích vector 2048 chiều cho mỗi ảnh.

Mã hoá dữ liệu (Encoding)

Mục tiêu:

- Trong giai đoạn tiền xử lý, encoding là quá trình chuyển đổi các dữ liệu đầu vào (thường có dạng rời rạc, phi cấu trúc) thành dạng vector số học nhằm mục đích:
 - Làm dữ liệu trở nên hiểu được đối với các thuật toán máy học hoặc khai phá dữ liệu.
 - Bảo toàn hoặc rút trích được những ngữ nghĩa và cấu trúc nội tại của dữ liệu
 - Chuẩn hoá định dạng biểu diễn giữa các loại dữ liệu khác nhau (cấu trúc, bán cấu trúc, phi cấu trúc)

4. Encoding âm thanh (Audio Data)

Trích xuất đặc trưng từ tín hiệu âm thanh theo thời gian để phục vụ phân tích hoặc nhận diện.

Các kỹ thuật chuyển đổi:

1. MFCC(Mel-Frequency Cepstral Coefficients):

- Chuyển đổi tín hiệu sang miền tần số (spectrogram) phản ánh đặc điểm thính giác con người.
- Đầu ra là ma trận (T, d), với T: số frame thời gian

2. Chroma, Spectrogram, Mel Spectrogram:

- Các dạng biến đổi tín hiệu phổ biến khác.

3. Pretrained Embedding:

- Wav2vec, HuBERT:
 - Học đặc trưng không giám sát từ waveform thô
 - Output: vector hoặc chuỗi vector đặc trưng

Mã hoá dữ liệu (Encoding)

Mục tiêu:

- Trong giai đoạn tiền xử lý, encoding là quá trình chuyển đổi các dữ liệu đầu vào (thường có dạng rời rạc, phi cấu trúc) thành dạng vector số học nhằm mục đích:
 - Làm dữ liệu trở nên hiểu được đối với các thuật toán máy học hoặc khai phá dữ liệu.
 - Bảo toàn hoặc rút trích được những ngữ nghĩa và cấu trúc nội tại của dữ liệu
 - Chuẩn hoá định dạng biểu diễn giữa các loại dữ liệu khác nhau (cấu trúc, bán cấu trúc, phi cấu trúc)

5. Encoding video (Video Data)

Biểu diễn nội dung động của chuỗi khung hình thành vector học được.

Các kỹ thuật chuyển đổi:

1. CNN + RNN/Transformer:

- Encode từng frame ảnh bằng CNN → tạo ra một sequence vector.
- Sequence được xử lý bởi RNN (LSTM/GRU) hoặc Transformer.

2. Clip-level Transformer:

- TimeSformer:
 - Áp dụng self-attention theo cả chiều không gian và thời gian.
- VideoMAE:
 - Học đặc trưng bằng cơ chế autoencoder, hiệu quả cao cho video không giám sát.

3. 3D CNN (C3D, I3D):

- Mở rộng CNN theo chiều thời gian để học đặc trưng động học.